

Optimisation combinatoire

2^e année du cycle supérieur — Spécialité SIQ

Problème de Bin Packing 1D

Méthodes approchées : Heuristiques de construction et d'amélioration

Réalisé par

BOUKAKIOU Rayan
KHERROUBI Mohamed El Amine
ZIADI Idriss Yacine
HIMEUR Idris
DIAR Adem Abdelhafidh

1 Introduction

Le bin packing unidimensionnel consiste à ranger des objets de tailles w_i dans des bacs de capacité C en minimisant le nombre de bacs utilisés. Ce problème étant NP-difficile, les méthodes exactes deviennent rapidement impraticables sur de grandes instances. Les **heuristiques** constituent alors une alternative naturelle : elles sacrifient la garantie d'optimalité au profit d'une exécution en temps polynomial, tout en produisant des solutions de qualité acceptable en pratique. Dans ce travail, nous présentons deux familles de méthodes approchées. Les **heuristiques de construction** — Next Fit, First Fit, Best Fit et Worst Fit, dans leurs variantes directe et décroissante — construisent une solution en une seule passe sur les objets. Les **heuristiques d'amélioration** — Relocation et Swap — partent d'une solution existante et tentent de réduire le nombre de bacs par des transformations locales itératives.

2 Formulation du problème

On dispose d'objets $i \in \{1, \dots, n\}$ de tailles $w_i \in \mathbb{N}$ et de bacs identiques de capacité $C \in \mathbb{N}$. Une solution est une partition des objets en groupes (bacs) telle que, pour chaque bac, la somme des tailles des objets affectés n'excède pas C . L'objectif est de **minimiser le nombre de bacs utilisés**. Une formulation classique en programmation linéaire en nombres entiers utilise des variables $x_{ij} \in \{0, 1\}$ indiquant si l'objet i est placé dans le bac j , et $y_j \in \{0, 1\}$ indiquant si le bac j est utilisé. On obtient alors le modèle suivant :

$$\min \sum_{j=1}^n y_j$$

sous les contraintes :

$$\sum_{j=1}^n x_{ij} = 1 \quad \forall i \in \{1, \dots, n\}, \quad \sum_{i=1}^n w_i x_{ij} \leq C y_j \quad \forall j \in \{1, \dots, n\}, \quad x_{ij} \in \{0, 1\}, y_j \in \{0, 1\}.$$

3 Heuristiques de construction

Les heuristiques de construction placent les objets un par un, dans un ordre fixé, selon une règle d'affectation déterministe. Chaque objet est assigné immédiatement et définitivement à un bac, sans retour en arrière. La complexité de ces méthodes est au plus $O(n^2)$ dans leur implémentation directe.

3.1 Next Fit (NF)

Next Fit est l'heuristique la plus simple. Un seul bac reste ouvert à tout instant. L'objet courant est placé dans ce bac s'il y tient ; sinon, le bac est fermé définitivement et un nouveau bac est ouvert. La règle d'affectation est donc sans mémoire : un bac fermé n'est plus jamais reconsidéré. Cette propriété confère à Next Fit une complexité $O(n)$ et une empreinte mémoire minimale, au prix d'une qualité de solution généralement plus faible. On

peut montrer que le nombre de bacs produit par NF vérifie $NF(I) \leq 2 \cdot OPT(I) - 1$ pour toute instance I , ce qui constitue sa garantie de performance.

3.2 First Fit (FF)

First Fit conserve l'ensemble des bacs ouverts. Pour chaque objet, on parcourt les bacs dans l'ordre de création et on place l'objet dans le *premier* bac où il tient. Si aucun bac ne peut l'accueillir, un nouveau bac est ouvert. En maintenant davantage d'options que Next Fit, First Fit exploite mieux la capacité résiduelle des bacs déjà ouverts. Son ratio de performance vérifie $FF(I) \leq \lfloor 1,7 \cdot OPT(I) \rfloor$ en théorie, et ses résultats pratiques sont généralement bien meilleurs.

3.3 Best Fit (BF)

Best Fit applique le principe du *meilleur ajustement* : pour chaque objet, on sélectionne le bac ouvert dont la capacité résiduelle après insertion serait la plus petite tout en restant non négative. Autrement dit, on cherche le bac dont l'espace restant est minimal après placement. Cette stratégie tend à concentrer les objets dans des bacs bien remplis, ce qui libère des bacs à forte capacité résiduelle pour de futurs objets volumineux. Best Fit partage le même ratio de performance asymptotique que First Fit ($BF(I) \leq \lfloor 1,7 \cdot OPT(I) \rfloor$) mais se distingue souvent favorablement sur des instances pratiques.

3.4 Worst Fit (WF)

Worst Fit adopte la stratégie inverse : chaque objet est placé dans le bac dont la capacité résiduelle après insertion serait la plus grande. L'intuition est de répartir la charge de manière équilibrée entre les bacs, en évitant de saturer prématurément certains d'entre eux. En pratique, cette répartition uniforme est souvent sous-optimale, car elle nuit au compactage et tend à produire davantage de bacs partiellement remplis que Best Fit ou First Fit.

4 Variantes décroissantes

Chacune des quatre heuristiques précédentes admet une **variante décroissante**, obtenue en triant les objets par taille décroissante avant de lancer la procédure de construction. L'idée est simple : les grands objets, plus contraignants à placer, sont traités en priorité lorsque les bacs sont encore vides et offrent le plus de flexibilité. Les objets plus petits, abordés en second, servent à combler les espaces résiduels.

En pratique, cette transformation améliore systématiquement la qualité de la solution. L'exemple le plus étudié est **First Fit Decreasing (FFD)**, dont on peut montrer que $FFD(I) \leq \frac{11}{9} OPT(I) + \frac{6}{9}$ pour toute instance I — un résultat classique dû à Johnson (1973). Ce ratio, nettement meilleur que celui de FF, explique pourquoi FFD est systématiquement utilisée comme heuristique de référence dans les méthodes exactes (notamment pour fournir une borne supérieure initiale dans un Branch & Bound).

Les variantes **Best Fit Decreasing (BFD)**, **Worst Fit Decreasing (WFD)** et **Next Fit Decreasing (NFD)** bénéficient du même avantage du tri, bien que leurs garanties théoriques diffèrent. BFD donne en général des résultats comparables à FFD. NFD améliore Next Fit mais reste plus faible que FFD ou BFD faute d'accès aux bacs fermés. WFD est rarement compétitive avec FFD ou BFD, malgré le tri initial.

Dans l'implémentation, les objets sont triés une seule fois en dehors des heuristiques et stockés dans `_items_in_descending`, ce qui évite tout surcoût de tri à chaque appel.

5 Heuristiques d'amélioration

Les méthodes d'amélioration partent d'une solution de départ — typiquement issue de FFD — et cherchent à réduire le nombre de bacs utilisés par des transformations locales répétées. Contrairement aux heuristiques de construction, elles opèrent sur la solution globale et peuvent modifier simultanément plusieurs bacs.

5.1 Relocation

La **relocation** cherche à *vider* des bacs peu chargés en redistribuant leurs objets dans les autres bacs. L'algorithme trie les bacs par charge croissante, puis tente, pour chaque bac source, de réaffecter tous ses objets un par un vers un autre bac existant qui peut les accueillir. L'opération n'est validée que si *tous* les objets du bac source ont pu être relocalisés : si un seul objet ne trouve pas de destination, la tentative est abandonnée et l'état initial est restauré. En cas de succès, le bac source devient vide et est supprimé, réduisant ainsi d'une unité le nombre total de bacs. Le processus se répète jusqu'à ce qu'aucune relocation ne soit possible.

Formellement, on cherche un bac s tel qu'il existe une affectation des objets $\{i \in s\}$ vers les bacs restants $\{b \neq s\}$ satisfaisant les contraintes de capacité. La relocation offre une amélioration garantie à chaque itération lorsqu'elle réussit, avec une complexité par itération en $O(k^2 \cdot m)$ où k est le nombre d'objets dans le bac source et m le nombre de bacs ouverts. La relocation est appliquée après FFD dans l'implémentation.

5.2 Swap

Le **swap** étend la relocation en autorisant des *échanges d'objets entre deux bacs*. L'algorithme commence par appliquer la relocation, puis itère une boucle d'amélioration : pour chaque paire de bacs (A, B) et pour chaque paire d'objets $(i \in A, j \in B)$, il teste si l'échange de i et j entre A et B est réalisable, c'est-à-dire si les charges résultantes restent dans la capacité C . Si l'échange est réalisable, la relocation est ensuite appliquée à l'état modifié. L'échange n'est conservé que si la relocation parvient à réduire le nombre total de bacs ; sinon, l'état est ignoré.

Ce couplage *swap + relocation* permet d'explorer un voisinage plus riche que la seule relocation : un échange peut débloquer une redistribution qui n'était pas possible avant, permettant ainsi d'atteindre des configurations inaccessibles par relocation seule. Le swap présente toutefois une complexité plus élevée, en $O(m^2 \cdot k^2)$ par itération de boucle externe,

et reste donc réservé aux instances de taille modérée.

6 Conclusion

Les **heuristiques de construction** — Next Fit, First Fit, Best Fit et Worst Fit — offrent une solution rapide en temps polynomial, dont la qualité croît avec la complexité de la règle d'affectation. Le **tri décroissant** améliore systématiquement les résultats de chaque variante, FFD constituant la référence pratique avec sa garantie $\frac{11}{9} \text{OPT} + \frac{6}{9}$. Les **méthodes d'amélioration** — relocation et swap — affinent une solution initiale par transformations locales itératives et permettent souvent d'atteindre l'optimum ou de s'en approcher, sans toutefois le garantir. La **relocation** est efficace et peu coûteuse ; le **swap**, plus exploratoire, offre de meilleures solutions au prix d'un temps de calcul plus élevé. Ces méthodes approchées constituent un compromis qualité-rapidité précieux, complémentaire aux méthodes exactes présentées dans le rapport précédent.